

package handlers

```
import (  
    "mime/multipart"  
    "net/http"  
    "os"  
    "strconv"  
    "strings"  
    "time"  
  
    "music-library-api/internal/models"  
    "music-library-api/internal/services"  
  
    "github.com/gin-gonic/gin"  
    "github.com/hajimehoshi/go-mp3"  
)
```

```
type TrackCreateRequest struct {  
    Artist    string          `form:"artist" binding:"required"`  
    Album     string          `form:"album"`  
    Genre     string          `form:"genre"`  
    ReleaseYear int         `form:"release_year"`  
    File      *multipart.FileHeader `form:"file" binding:"required" // file mp3`  
}
```

```
type UpdateTrackRequest struct {  
    Title     string `json:"title" binding:"required"`  
    Artist    string `json:"artist" binding:"required"`  
    Album     string `json:"album"`  
    Genre     string `json:"genre"`  
    ReleaseYear int `json:"release_year"`  
    Duration  int  `json:"duration"`  
    MP3URL    string `json:"mp3_url"`  
}
```

```
type TrackHandler struct {  
    service services.ITrackService  
}
```

```
func NewTrackHandler(service services.ITrackService) *TrackHandler {  
    return &TrackHandler{service: service}  
}
```

// GET /tracks

```
func (h *TrackHandler) GetTracks(c *gin.Context) {  
    page, _ := strconv.Atoi(c.DefaultQuery("page", "1"))  
    limit, _ := strconv.Atoi(c.DefaultQuery("limit", "10"))
```

```
    tracks, err := h.service.GetTracks(page, limit)  
    if err != nil {  
        c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})  
        return  
    }
```

```
    c.JSON(http.StatusOK, gin.H{"page": page, "limit": limit, "data": tracks})  
}
```

```

// GET /tracks/:id
func (h *TrackHandler) GetTrackByID(c *gin.Context) {
    id := c.Param("id")
    track, err := h.service.GetTrackByID(id)
    if err != nil {
        c.JSON(http.StatusNotFound, gin.H{"error": "track not found"})
        return
    }
    c.JSON(http.StatusOK, track)
}

// POST /tracks
func (h *TrackHandler) CreateTrack(c *gin.Context) {
    var req TrackCreateRequest

    // Bind multipart/form-data
    if err := c.ShouldBind(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    // check file type
    if !strings.HasSuffix(strings.ToLower(req.File.Filename), ".mp3") {
        c.JSON(http.StatusBadRequest, gin.H{"error": "only mp3 files are allowed"})
        return
    }

    // 1. save file
    dst := "uploads/" + req.File.Filename
    if err := c.SaveUploadedFile(req.File, dst); err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "failed to save file"})
        return
    }

    // 2. open file
    f, err := os.Open(dst)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "cannot open file"})
        return
    }
    defer f.Close()

    // 3. decode mp3 to get duration
    decoder, err := mp3.NewDecoder(f)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "cannot decode mp3"})
        return
    }

    // go-mp3 decodes to 16-bit (2 bytes) stereo (2 channels).
    // Therefore, each sample uses 4 bytes (2 bytes/sample * 2 channels).
    // Total Samples = decoder.Length() / 4
    // Duration (seconds) = Total Samples / Sample Rate

```

```
totalBytes := float64(decoder.Length())
sampleRate := float64(decoder.SampleRate())
```

```
durationSeconds := (totalBytes / 4.0) / sampleRate
```

```
duration := time.Duration(durationSeconds * float64(time.Second))
```

```
// 4. track object
```

```
track := &models.Track{
    Title:    req.File.Filename,
    Artist:   req.Artist,
    Album:    req.Album,
    Genre:    req.Genre,
    Duration: int(duration.Seconds()),
    URL:      dst,
    ReleaseYear: req.ReleaseYear,
}
```

```
// 5. save track to DB
```

```
if err := h.service.CreateTrack(track); err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"error": "failed to save track"})
    return
}
```

```
c.JSON(http.StatusCreated, track)
}
```

```
// PATCH /tracks/:id
```

```
func (h *TrackHandler) UpdateTrack(c *gin.Context) {
    id := c.Param("id")
```

```
    track, err := h.service.GetTrackByID(id)
    if err != nil {
        c.JSON(http.StatusNotFound, gin.H{"error": "track not found"})
        return
    }
```

```
    var req UpdateTrackRequest
```

```
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }
```

```
    if req.Title != "" {
        track.Title = req.Title
    }
```

```
    if req.Artist != "" {
        track.Artist = req.Artist
    }
```

```
    if req.Album != "" {
        track.Album = req.Album
    }
```

```
    if req.Genre != "" {
        track.Genre = req.Genre
    }
```

```

if req.ReleaseYear != 0 {
    track.ReleaseYear = req.ReleaseYear
}
if req.Duration != 0 {
    track.Duration = req.Duration
}
if req.MP3URL != "" {
    track.URL = req.MP3URL
}

if err := h.service.UpdateTrack(track); err != nil {
    c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
    return
}

c.JSON(http.StatusOK, track)
}

// DELETE /tracks/:id
func (h *TrackHandler) DeleteTrack(c *gin.Context) {
    id := c.Param("id")
    track, err := h.service.GetTrackByID(id)
    if err != nil {
        c.JSON(http.StatusNotFound, gin.H{"error": "track not found"})
        return
    }

    if err := h.service.DeleteTrack(track.ID.Hex()); err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
        return
    }

    c.JSON(http.StatusNoContent, nil)
}

// GET /tracks/search
func (h *TrackHandler) SearchTracks(c *gin.Context) {
    query := c.Query("q")
    page, _ := strconv.Atoi(c.DefaultQuery("page", "1"))
    limit, _ := strconv.Atoi(c.DefaultQuery("limit", "10"))

    tracks, err := h.service.SearchTracks(query, page, limit)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
        return
    }

    c.JSON(http.StatusOK, gin.H{"page": page, "limit": limit, "data": tracks})
}

```